

Fine-Grained Vision Foundation Models for Automated Wind Turbine Inspection: Supporting Materials

Alvyn Shibu
40356093

1. Extended Literature and Context

1.1 Scope and Method

The research paper's literature review is limited to approximately one page and is focused on the work that directly supports the contribution. The purpose of this section is to expand that review across five areas: blade inspection datasets and detection techniques, conventional detection architectures, vision foundation models, foundation models in industrial inspection, and vision-language approaches to anomaly detection. Sources were identified using Google Scholar, IEEE Xplore and ScienceDirect. The term combinations used were *wind turbine blade*, *defect detection*, *UAV inspection*, *foundation model*, *vision transformer* and *anomaly detection*, supplemented by forward and backward citation tracing from the papers cited in the research paper. Priority was given to peer-reviewed work published from 2021 onwards, with earlier work included only where it establishes context that later work builds upon.

1.2 Blade Inspection: Data, Methods and Problems

Automated blade inspection research divides into two areas: work that provides data and work that provides architecture. These two areas have developed at different rates, and the resulting imbalance between them is the field's defining characteristic.

Dataset contributions. Shihavuddin et al. [1] produced the DTU-Drone dataset, which remains the most widely used resource in automated blade inspection. Four variants of Faster R-CNN were tested on it, with the best achieving 81.1% mAP using Inception-ResNet-V2. The influence of DTU-Drone is far greater than its size would suggest, largely because so few alternatives exist. Foster et al. [2] continued detection work using drone footage from the same domain. Gohar et al. [3] subsequently addressed the dataset's greatest limitation by releasing a set of standardised five-class annotations, and later proposed slice-aided hyper-inference for analysing ultra-high-resolution imagery [4]. Yang et al. [5] developed Blade30, a dataset focused on image stitching rather than detection. More recently, Ji et al. [6] released WTBD with six defect categories and a defined split protocol, and Zhang et al. [7] released WTBS2025 spanning nine categories.

Architectural contributions. A large body of literature focuses on modifying the YOLO family for blade inspection. Yu et al. [8] combined CBAM attention and BiFPN within YOLOv8. Zou et al. [9] developed DCW-YOLO for surface damage detection. He et al. [10] proposed an adaptive multi-scale detection method. Li et al. [11] modified YOLOv5 for operational condition monitoring from UAV imagery. Guo et al. [12] applied deep convolutional networks to blade damage identification, and Li et al. [13] combined an improved SegFormer with pixel matching to measure defect size. Dwivedi et al. [14] addressed blade and solar panel defects jointly using an attention-based model.

Alternative sensing modalities. Some researchers have pursued better data rather than better architectures. Jia and Chen [15] combined optical and thermal video for near-real-time segmentation. Memari et al. [16] combined multi-spectral RGB and thermal imaging to detect anomalies on small turbines. Rizk et al. [17] analysed hyperspectral imaging data using 3D convolutional networks. Deng et al. [18] took the opposite approach and applied classical digital image processing.

The unaddressed problem. Three shared aims run through this literature. Firstly, almost every study evaluates against a single dataset, so how well a model generalises across sources is rarely examined. Secondly, each dataset uses a different taxonomy, which makes merging difficult and cross-study comparison unreliable; Gohar et al. [19] highlights this issue specifically in their review. Thirdly, and least acknowledged, annotation quality is assumed rather than verified. None of the studies investigated here reported checking the accuracy of the annotations they trained on. The annotation audit carried out during this project found that two categories within a dataset published as a high-quality image dataset contained significant labelling errors, which indicates that this assumption is not always safe and that annotation verification should become standard practice in the field.

The architectural improvements proposed in this literature: deeper backbones, attention modules, sliced inference, all address data scarcity by modifying the model. None considers whether the source of the inputted data could be altered instead, and that question forms the basis of the present study.

1.3 Conventional Detection Architectures

The single-stage detection family provides the baselines used in this work. The YOLO family has converged on an anchor-free, decoupled-head design: YOLOv8 [20] established the modern form, YOLOv9 [21] introduced adjustable gradient information to address information loss in deeper networks, and YOLOv10 [22] removed the non-maximum suppression stage to allow end-to-end training. RT-DETR [23] takes a different route, adapting the DETR set-prediction formulation for real-time reasoning and reporting higher COCO accuracy than comparable YOLO variants.

All four are designed for object detection in general, where training data is plentiful and objects occupy a substantial portion of the frame. Both assumptions do not hold for blade inspection. Datasets are small, and defects are frequently only a few tens of pixels across within an image several thousand pixels wide. This mismatch motivates the small-object techniques common in the domain literature, and it is also the condition under which a strong, pretrained representation would be expected to provide the greatest benefit.

1.4 Vision Foundation Models and Plain-Backbone Detection

Two developments make the present approach possible.

Plain backbones for detection. Li et al. [24] showed that a non-hierarchical vision transformer can be used as a detection backbone and only requires a simple feature pyramid constructed from a single-scale feature map rather than a network specifically designed for detection. This breaks the link between backbone quality and detection-specific design and is the architectural example for the model described here.

Agglomerative pretraining. Self-supervised and language-supervised pretraining have produced several strong but specialised backbones. CLIP [25] learns a joint vision-language representation with strong zero-shot classification performance. DINO [26] and its successor DINOv2 [27] generate robust patch-level embeddings that transfer well to dense prediction. SAM [28] provides fine-grained segmentation ability. Ranzinger et al. [29] recognised that these capabilities are complementary and refined all three into AM-RADIO, an agglomerative student model that outperforms each of its teachers across most benchmarks while retaining both zero-shot and dense-prediction behaviour. Heinrich et al. [30] refined this further with RADIOv2.5, using multi-resolution training

and balanced teacher losses to address resolution-dependent mode switching and teacher imbalance.

The agglomerative approach is particularly suited to defect detection because the task has a dual nature. Locating small defect regions is a dense localisation problem, best served by DINOv2-style features, while distinguishing between visually similar defect types is a semantic discrimination problem, best served by CLIP-style features. A single-teacher backbone would without doubt trade one against the other.

1.5 Foundation Models in Industrial Inspection

The application of foundation models to industrial defect detection has developed largely within the anomaly detection space, which differs from the present work in an important respect, it treats defects as deviations from a learned idea of ‘normal’ rather than as classes to be detected.

Memory-bank methods. SPADE [31] and PaDiM [32] introduced multi-scale patch correspondence and Gaussian modelling of patch features. PatchCore [33] combined these ideas into a widely adopted approach: extract dense patch embeddings from a pretrained backbone, compress them through coreset selection, and score anomalies by nearest-neighbour distance. GraphCore [34] extended this with graph-based memory. These methods are appealing because they require no training, but Reiss et al. [35] showed that their performance depends heavily on the quality of the underlying feature encoder.

Foundation-model backbones. That dependence motivated substituting stronger encoders. AnomalyDINO [36] builds a patch-memory pipeline on DINOv2 and achieves state-of-the-art few-shot performance on MVTec-AD [37] with competitive results on VisA [38]. RadioCore [39] extends the idea to an agglomerative backbone, using RADIOv2.5 features in a multi-scale memory bank with k-nearest-neighbour matching. Iqbal et al. [40] pursued a reconstruction-based alternative using multi-scale feature reconstruction.

Distinction from the present work. All these methods are training-free: the backbone is used as a fixed feature extractor and no gradients pass through it. The current work fine-tunes the backbone alongside a trainable detection head, which is a categorically different proposition. The benchmarks used throughout, bottles, capsules, printed circuit boards, also resemble web-image pretraining distributions far more closely than aerial imagery of weathered composite surfaces, so the domain shift these methods face is considerably smaller than the one examined here.

1.6 Vision-Language Models for Defect Description

The secondary aspect of this project draws on similar literature applying vision-language models to industrial inspection. WinCLIP [41] showed that CLIP embeddings enable zero-shot anomaly classification through a structural ensemble of attribute words and prompt structures, combined with window-based feature aggregation for localisation. APRIL-GAN [42] and MuSc [43] developed related zero-shot approaches, while AnomalyCLIP [44] proposed object-agnostic prompt learning to reduce the category-specific engineering each new prompt otherwise requires. Zajec et al. [45] studied the corresponding few-shot problem in manufacturing.

All these studies report strong performance on manufactured-object benchmarks, but all were conducted solely within that domain. The replication presented in Section IV-E of the research paper shows that the reported performance does not transfer to aerial

blade imagery, which is consistent with the domain-shift argument above and supports it directly.

1.7 Synthesis and Gap

Three concerns come from this review. Blade inspection research is primarily concerned with architectural refinement, and few studies address whether a model generalises across sources or whether the annotations used to train it are accurate. Vision foundation models have shown somewhat promising results in industrial inspection, but almost exclusively in training-free settings where domain shift between source and target is limited. No study combines the two approaches: fine-tuning an agglomerative foundation model with a trainable detection head for blade defect detection and comparing its performance against contemporary conventional detectors under both single-dataset and merged cross-dataset training.

The research paper addresses that combined gap, which is shown to be a genuine one through this review.

2. Project Life Cycle

2.1 Overview and Methodology

The project followed an iterative, experimental cycle rather than a straightforward waterfall. The output of each stage shaped the design of the next: initial baseline results determined the hyperparameter search space, per-dataset results motivated the cross-dataset design, and measurement faults discovered during evaluation forced revisions to the training pipeline itself. This section describes each stage and notes where the plan changed and why.

Supervision took the form of regular progress updates against the agreed objectives. Two supervisory interventions shaped the work materially. Early in the project, it was established that the work should demonstrate whether foundation models can outperform smaller conventional models when appropriately fine-tuned. Later, a requirement was set that this comparison be conducted in both per-dataset and merged cross-dataset settings, supported by lots of experimentation.

2.2 Planning and Scoping

The initial scope established the research question, identified AM-RADIO as the candidate foundation model, and selected the YOLO and RT-DETR families as the conventional comparison. Three decisions taken at this stage influenced everything that followed.

Choice of backbone. AM-RADIO was chosen over single-teacher models such as DINOv2 because blade defect detection requires both dense localisation and semantic discrimination between visually similar categories. The full rationale is given in Section 5.1.

Choice of task framing. The problem was framed as supervised object detection rather than unsupervised anomaly detection, which is the more common framing for foundation-model work in industrial inspection (Section 1.5). Supervised framing was chosen because the available datasets carry class labels, and because the model needs defects categorised rather than just flagged.

Choice of comparison protocol. Benchmarking against eighteen conventional architectures rather than one was decided early, to prevent the result being attributed to a poorly selected baseline.

Three risks were identified at planning stage: limited availability of annotated blade datasets, computational constraints associated with a 651-million-parameter backbone, and the absence of a common taxonomy across sources. All three happened. The first and third initially proved more damaging than imagined. The second was managed successfully but shaped the experimental design throughout.

2.3 Data Acquisition and Preparation

Five datasets met the criteria for use: Blade30, the Gohar re-annotation of DTU-Drone, WTBs2025, WTBD, and Yae, derived from the dataset of Liu and Liu. Initially, only three datasets were found (Blade30, Gohar and WTBD), however the lack of data made generalisation extremely difficult, especially in certain individual classes which were severely underrepresented. WTBs2025 and Yae were then found to fill that gap.

Preparing these datasets required considerably more effort than predicted. Each arrived in a different annotation format and used a different defect taxonomy, ranging from two to nine categories with no consistency in naming. A single six-category taxonomy was therefore defined, and a single conversion pipeline written to transform all five sources into it. Using one pipeline rather than per-dataset scripts was intended to ensure that conversion differences could not introduce contradicting factors into the cross-dataset comparison.

The greatest unanticipated effort at this stage was the annotation audit described in Section 3.4. Examining a subset of annotated instances revealed that one dataset contained substantial annotation error, which required manual review of 1,085 images in a single category and the exclusion of seven of its nine categories. This work was not planned for, and it consumed a significant portion of the schedule, but it prevented training on unreliable labels, underrepresented classes and produced a finding reported in the research paper.

2.4 Implementation

Implementation worked across three components.

Conventional baselines. Built using the Ultralytics framework with minimal custom code, starting from official COCO-pretrained checkpoints. The framework's original training loop, augmentation and evaluation were left unmodified, so that the baseline reflects what a developer would realistically achieve rather than a deliberately weakened comparison.

Foundation-model detector. Written as custom code. It comprises a wrapper exposing AM-RADIO ViT-H/16 features, three trainable projection blocks constructing a P3/P4/P5 pyramid, and the Ultralytics FPN/PAN neck and Detect head reused so that the two families differ only in backbone and adaptation strategy. A configurable freezing mechanism controls how many final transformer blocks receive gradients.

Vision-language pipeline. Developed independently. Cropped regions were extracted from all five datasets with padding and a minimum dimension threshold, negative examples were sampled from image corners, the resulting examples were converted to an instruction format, and LoRA fine-tuning was performed using Unsloth.

Development was iterative rather than specified-then-built. Several architectural decisions, GroupNorm in place of BatchNorm, the EMA implementation, and gradient clipping thresholds, were made in response to observed training behaviour rather than

selected in advance. Section 5.2 gives the reasoning behind these choices and considers, with hindsight, which could have been anticipated.

2.5 Experimentation

Experimentation ran in two arms.

Per-dataset arm. For each of the five datasets, eighteen conventional architectures were trained and the best retained, and a nine-configuration grid search was run over the two hyperparameters controlling backbone adaptation. This produced 45 foundation-model training runs and 90 baseline runs.

Cross-dataset arm. The training splits of four datasets were pooled into a single 4,243-image set, with Blade30 excluded entirely. Both families were trained on the pooled data and evaluated on the merged validation split and on each source's individual test set.

Compute management. All parts ran on Google Colab using a single NVIDIA A100-SXM4 40GB GPU. Each merged-dataset epoch took approximately 7.5 minutes, with an evaluation pass every ten epochs adding a further 14 minutes, so a 100-epoch run required roughly 13 hours. Checkpoints were written to Google Drive at ten-epoch intervals so that a session disconnection would not lose the run. This cost shaped the experimental design directly: the cross-dataset configuration was carried forward from per-dataset results rather than independently re-searched, because a nine-configuration grid at merged scale was not affordable.

2.6 Evaluation and Iteration

Evaluation was a continuous process rather than a final phase. Three separate faults were identified during this stage, each requiring a return to implementation.

The first was an exponential moving average evaluation lag, caused by the custom EMA implementation not matching the framework's methodology, which understated performance by 0.29 mAP at one checkpoint. The second was batch normalisation instability at batch size 4, producing a mean absolute discrepancy of 0.28 to 0.38 between training and evaluation behaviour and an apparent mid-training collapse; this was resolved by replacing BatchNorm with GroupNorm. The third was a checkpoint serialisation fault, in which a saved model failed to reproduce the validation score recorded in its own training log. This was detected by reloading the checkpoint and re-evaluating it rather than relying on the logged score alone.

The third fault required the merged experiment to be re-run in full. The re-run incorporated the GroupNorm change together with a verification step and produced a checkpoint that reproduced its logged metric to within 0.003 mAP.

3. Verification and Validation

3.1 Approach

Verification was conducted at five levels: data origin, label correctness, data quality, the training process, and saved artefacts. The methodology was only partially planned. It mainly developed in response to faults identified during the project, each of which was only detectable by testing something that had previously been assumed correct. The resulting principle, that any value the pipeline reports should be independently reproducible from the artefacts it produces, is the project's main methodological lesson and is discussed further in Section 5.3.

3.2 Data Provenance and Cross-Dataset Contamination

Because the cross-dataset experiment trains on pooled data and evaluates on individual test sets, an image appearing in more than one source would contaminate the evaluation. Several blade datasets derive from overlapping UAV collections, so this was a tangible risk rather than a theoretical one.

Every image across all five datasets was hashed using the imagehash library. The resulting hashes were compared pairwise between datasets at a distance threshold permitting both exact matches and near-duplicates. Matches were recorded with their distance values and reviewed individually, sorted by confidence. Confirmed duplicates were quarantined rather than deleted, so that the decisions taken remain auditable. Perceptual hashing is established practice elsewhere in computer vision, but it does not appear in the blade inspection literature surveyed in Section 1. Omitting it would leave any cross-dataset generalisation claim vulnerable to undetected overlap between sources.

3.3 Label Correctness

Two independent methods were used.

Round-trip provenance verification. Rather than inspecting converted labels in isolation, each converted image was matched back to its raw source using its perceptual hash. The correct class set was then extracted directly from the raw annotation file, in LabelMe JSON format for Blade30, Pascal VOC XML for WTBD and Yae, and per-folder YOLO labels for WTBS2025, and compared against the classes written by the conversion pipeline. For every dataset, the procedure reports how many images were checked, how many could not be matched to a raw source, and every class mismatch with both the expected and written values.

This process was carried out across all splits for Blade30, Yae and WTBS2025. It validates the conversion pipeline against ground truth rather than against itself and detects both conversion faults and errors in the source annotations. Where mismatches appeared, each instance was traced back to its raw source and examined to establish whether the mismatches followed a systematic pattern.

Class index mapping. Index-to-class mappings were verified against the metadata distributed with each dataset, including Roboflow classes.txt and data.yaml files where present, rather than being inferred from folder names. Silent index misalignment produces a model that trains successfully and evaluates meaninglessly, causing failure with no visible symptoms.

Visual class sampling. Labelled instances were rendered per class in grid form for each dataset, so that categories could be inspected for internal consistency and for agreement with the shared taxonomy.

3.4 Data Quality and the WTBS2025 Audit

Visual inspection revealed that one dataset's annotations disagreed with its image content at a rate that made it unusable as released.

WTBS2025 spans nine defect categories across 7,544 images. Three categories had no equivalent in the shared taxonomy and were excluded at the outset. The lightning strike category was excluded because it combines 57 real photographs with 228 GAN-synthesised images that could not be reliably separated, a limitation that the dataset's own authors document. The remaining categories were checked visually, and sampled crack and erosion annotations were found to be incorrect at rates of approximately 80%

and 40% respectively, in the sense that the labelled region did not contain the stated defect. Both categories were therefore excluded.

Of the remaining categories, only coating detachment could be retained, and then only after exhaustive manual review. An interactive review tool was written to display each candidate image alongside its annotation for acceptance or rejection, with files pre-filtered to those containing the target class. All 1,085 images were reviewed individually and 161 were approved. The approved filename list was written to storage so that the decision set is reproducible, rather than dependent on a particular notebook session. Contamination, which sampling found reliable at approximately 90%, was kept entirely. Two categories and 933 images were kept from nine categories and 7,544 images. This is a significant rejection rate for a dataset published as a high-quality resource, and it demonstrates clearly that annotation verification should form part of any research effort using image-based datasets rather than overreliance on the validity of the source.

3.5 Training Process Verification

Model selection parity. The custom EMA implementation used in the foundation-model pipeline initially differed from the framework's approach, producing a value 0.29 mAP lower at one checkpoint. It was revised to use the same decay rate and time constant as the framework, so that both families select checkpoints under equal criteria.

Normalisation stability. Backbone memory constrains the batch size to four. At that size, batch normalisation accumulates unstable running statistics. Measuring the difference between training-mode and evaluation-mode behaviour showed a mean absolute discrepancy of 0.28 to 0.38, which produced an apparent mid-training collapse that was a measurement artefact rather than a genuine model failure. Replacing batch normalisation with group normalisation eliminated the discrepancy.

Gradient behaviour. Pre-clipping gradient norms were measured and found to range between approximately 100 and 500 against a clipping threshold of 10.0, indicating that clipping was active on virtually every iteration. This was confirmed at both batch size 4 and batch size 8, which establishes that per-step update magnitude is determined by clipping regardless of batch size and therefore bounds the practical impact of the effective batch size asymmetry between the two pipelines.

Configuration isolation. An earlier version of the freezing mechanism set parameters to trainable without resetting them, so the number of unfrozen blocks accumulated across configurations sharing a common backbone object. The revised mechanism returns the backbone to a fully frozen state before each configuration is run.

3.6 Artefact Verification

This level was added in due to a failure that none of the previous checks would have detected.

A merged dataset run reported a best validation score of 0.8306 mAP@0.50, with a consistent learning curve across 100 epochs. When the saved checkpoint was then loaded for per-test-set evaluation, it produced 0.0438, with a maximum class confidence of 0.097 on annotated defect images, comparable to its response to randomly generated input. The training log and the saved weights described different models.

The problem was found by elimination. State dictionary keys matched completely, normalisation buffers were finite and correctly built up, box decoding was confirmed against the raw output format, and the run's per-epoch history was recovered and confirmed to show genuine learning. The fault was found to be checkpoint selection and

serialisation under the batch normalisation architecture that run had used, which has since been replaced.

The fix was to implement a verification step into the training loop. After each checkpoint is written, the file is reloaded into a fresh model, re-evaluated, and the reloaded score compared against the logged score. The re-run reproduced its logged metric to within 0.003 mAP, with raw and EMA weights agreeing to within 0.0001 at convergence.

The lesson is that a training loop's reported metric is an assertion about an in-memory object rather than about the file on disk. Equivalence between the two must be demonstrated, not assumed.

3.7 Evaluation Protocol Validation

Both model families were trained to their own convergence under identical early-stopping criteria rather than a fixed epoch budget, evaluated at the same intersection-over-union thresholds through the same metric implementation, and selected on the same criterion.

Performance is reported at two IoU thresholds rather than one. A plain vision transformer backbone and a purpose-designed convolutional detector might differ in localisation quality specifically, and reporting both thresholds makes any such difference visible rather than concealing it behind a single figure.

WTBD provides no independent test split, so its validation split was used for evaluation throughout. Its results are therefore less strictly held out than those of the other four datasets.

Two aspects of the evaluation remain unvalidated. All results are produced from single training runs per configuration, and checkpoint-to-checkpoint variation reached 0.15 mAP on the smaller datasets, which exceeds the margin recorded on the smallest dataset; multi-seed evaluation would be required to establish that particular result with confidence. In addition, the merged conventional baseline was a single architecture rather than the best of the eighteen-architecture sweep used per dataset.

4. Tools Used to Support Development

4.1 Compute and Execution Environment

Google Colab (NVIDIA A100-SXM4 40GB). All training and evaluation were performed in this environment. Colab was chosen because a 651-million-parameter backbone at 1024×1024 input exceeds the memory capacity of a typical consumer GPU, and it provided A100 access without requiring formal cluster provisioning.

Its limitations affected the project more than those of any other tool. Sessions disconnect without warning, so every long-running process required checkpoint logic.

Local storage under /content is purged on disconnection, so datasets had to be re-copied. A 100-epoch merged-dataset run required approximately 13 hours, at 7.5 minutes per epoch plus a 14-minute evaluation pass every ten epochs. This cost restricted the experimental design directly: the cross-dataset hyperparameter configuration was carried forward from per-dataset results rather than searched from scratch, because a nine-configuration grid at merged scale was not affordable.

Three mitigations were implemented in response. Training loops check whether a local dataset copy exists and re-copy it automatically from Drive if not, rather than allowing the framework to fall back silently to a default dataset, which would produce meaningless results without raising an error. Completed runs are recorded in a JSON log

and skipped on re-execution. Partially completed runs are detected by counting rows in the framework's results.csv and resumed from the last checkpoint rather than restarted from scratch.

Google Drive. Persistent storage for datasets, checkpoints and results, mounted into the Colab runtime. It was chosen for durability across sessions rather than for performance.

4.2 Detection Frameworks

Ultralytics. Used to establish the baseline results and partly, the foundation-model. It provided the eighteen architectures benchmarked (YOLOv8n/s/m/l, YOLOv9c/e/s/m, YOLOv10n/s/m/l, YOLO11n/s/m/l, RT-DETR-l/x), each from official COCO-pretrained checkpoints, together with its native training loop, augmentation, evaluation and non-maximum suppression.

Ultralytics was selected for two reasons. It is the standard choice for a developer to employ, so the baseline provides a genuine comparison rather than an artificially weak one. Its C3k2, Conv and Detect modules could also be reused within the foundation-model architecture, so that the two families share detection logic and differ only in backbone and adaptation strategy, which is the difference being studied.

One framework-specific adjustment was necessary. RT-DETR required different hyperparameters from the YOLO family, using a classification loss weight of 1.2 rather than 2.0 and an initial learning rate of 1×10^{-5} rather than 1×10^{-2} , showing its different loss formulation. Using default YOLO everywhere would have created an incorrect show of weakness in the strongest transformer-based baseline.

A key limitation is that Ultralytics accumulates gradients to a nominal batch size of 64 by default, whereas the custom foundation-model pipeline does not. This inconsistency is mentioned in the research paper.

PyTorch. Used directly for the custom foundation-model training loop, the freezing mechanism, the exponential moving average, mixed-precision training with gradient scaling, and checkpoint serialisation.

torch.hub. Used to retrieve AM-RADIO ViT-H/16 checkpoints, selected because it is the distribution channel the model's authors provide.

torchmetrics. Used to calculate mean average precision within the foundation-model pipeline, providing an application independent of Ultralytics so that both families are evaluated through a comparable metric.

4.3 Data Preparation and Verification

imagehash. Perceptual hashing, used for two purposes: detecting duplicate and near-duplicate images across datasets, which would otherwise contaminate cross-dataset evaluation, and matching converted images back to their raw sources so that label correctness could be verified against the original annotations. The second use is what made the round-trip verification described in Section 3.3 possible. Exact file hashing was considered as an alternative, but re-encoded or resized copies of the same image produce different byte-level hashes while retaining very similar perceptual hashes.

ipywidgets. Used to build the interactive review tool for the WTBs2025 coating detachment audit, displaying each candidate image with its annotation for acceptance or rejection. Reviewing all 1,085 images would have been impractical without such a tool, which also wrote the approved filename list to storage so that the decision set is reproducible rather than confined to a single session.

PIL and matplotlib. Image loading, rendering annotations over source images for visual verification, per-class sample grids, training curve plots, and the qualitative detection figure.

PyYAML, pandas and shutil. Dataset configuration files; reading framework log files to determine resumption points; and dataset copying with retry logic, since Drive operations occasionally fail quickly under load.

4.4 Vision-Language Tooling

Hugging Face Transformers. Model loading and inference for BLIP-2, in both OPT and FlanT5-XL configurations, and Qwen2.5-VL-3B-Instruct during zero-shot evaluation.

OpenCLIP (LAION-400M). Used to replicate WinCLIP with the same backbone and compositional prompt ensemble described in the original publication, so that the replication tests the published method rather than a variation.

Unsloth with LoRA. Used for supervised fine-tuning of Qwen3-VL-2B-Instruct, updating around 1.1% of its 2.15 billion parameters. This approach was chosen because full fine-tuning of a 2B-parameter vision-language model would have exceeded the memory available, while Unsloth's FastVisionModel provides memory-efficient training through a straightforward interface.

scikit-learn. Used to compute AUROC for the WinCLIP replication, consistent with the evaluation methodology of the original publication.

This section required its own session recovery mechanism. Adapters were written to drive immediately after training, and a reload path was implemented so that a disconnection mid-evaluation did not require retraining.

4.5 Development and Version Control

Jupyter and Colab notebooks. Development was organised into separate notebooks by stage: dataset conversion, dataset verification, per-dataset baseline evaluation, per-dataset foundation-model training, cross-dataset experimentation, and vision-language experimentation. Separating them kept the long-running training notebooks isolated from the exploratory verification work, so that a crash in one did not affect the others. One significant disadvantage of the notebook format is that state continues implicitly between cells, so a variable defined in a cell that has since been deleted remains live in the session. This contributed directly to the difficulty of diagnosing the checkpoint fault described in Section 3.6, where an evaluation cell referenced a model class defined in a deleted cell. Moving code into version-controlled modules imported by the notebooks would remove this class of problem entirely.

GitLab. Version control and submission.

4.6 Summary of Tool-Driven Constraints

Three constraints that lead back to tooling shaped the results reported in the research paper. Colab's session limits and per-epoch cost determined that the cross-dataset configuration was carried forward rather than re-searched. Backbone memory at 1024×1024 input constrained the batch size to four, which caused the batch normalisation instability that motivated the switch to group normalisation. And Ultralytics' default gradient accumulation, absent from the custom pipeline, produced the effective batch size asymmetry disclosed as a limitation.

5. Critical Appraisal

5.1 Divergence from the Original Specification

The delivered project differs from the original specification in four respects. Each divergence was deliberate, and each is justified below.

Backbone: DINOv3 to AM-RADIO. The specification proposed DINOv3. AM-RADIO was substituted following discussion with the supervisor, on the grounds that defect detection requires two capabilities simultaneously that no single-teacher backbone supplies. DINOv3 yields strong structural patch representations, but blade defect detection also requires precise boundary delineation, such as tracing the perimeter of a fracture or the irregular edge of coating loss, along with semantic discrimination between visually similar categories. An agglomerative model distills multiple specialist teachers into a single encoder: SAM contributes segmentation-quality boundary sensitivity, DINOv2 contributes dense structural features, and CLIP contributes open-vocabulary semantic grounding. Since the published evidence is that the agglomerative student exceeds each individual teacher on most benchmarks [29], the substitution is not a compromise between them but an improvement over any one of them for a task requiring all three.

Output: visual encoder to detector. The specification originally described the deliverable as a strong visual encoder. This was reframed during implementation because a visual encoder alone cannot perform the task the project sets out to evaluate. AM-RADIO alone emits abstract feature representations and has no concept of a bounding box. Producing something capable of locating and classifying defects required constructing an entirely separate architecture on top of it: three trainable projection blocks mapping constant-resolution patch features into a P3/P4/P5 pyramid, a feature pyramid and path aggregation network, and a detection head with its associated loss. That construction was not optional, and it is not deviation from the specification's intent but the necessary consequence of evaluating an encoder on a detection task.

Modality: ultra-high-resolution video to still images. The specification proposed fine-grained analysis of ultra-high-resolution drone video. This proved infeasible within the project's compute and data constraints, for reasons that became clear working with just images.

A single 1024×1024 forward pass through a 651-million-parameter backbone constrained the batch size to four on a 40GB A100, and a 100-epoch run on 4,243 images required approximately 13 hours. Video at inspection-grade resolution multiplies this cost in three ways: frame count, since even short inspection sequences contain thousands of frames; resolution, since ultra-high-resolution footage would require tiled inference rather than a single downsampled pass; and temporal modelling, which would add architecture and parameters beyond the per-frame detector. Beyond compute, no annotated blade inspection video dataset with frame-level defect boxes was identified during the literature search described in Section 1 and constructing one would have consumed the entire project.

Training strategy: hybrid supervised and self-supervised to purely supervised. The specification proposed a hybrid strategy implementing dense visual supervision and synthetic data, whereas the delivered work is purely supervised. Two factors drove this. Firstly, the datasets acquired carry class labels and bounding boxes, so supervised training uses the available signal directly. Secondly, and more inevitably, the annotation audit described in Section 3.4 consumed a substantial portion of the schedule, and adding a self-supervised pretraining stage and synthetic data generation on top would have left insufficient time for the experimental comparison identified as the project's

core requirement. The decision was to establish a rigorous supervised comparison rather than a partial hybrid one.

Assessment of the divergences. The first two are justified on technical grounds and would have been the right choices regardless of scheduling. The third is forced by resource constraints that were not anticipated. The fourth is a reduction in scope linked to unplanned data quality work. Taken together, the delivered project answers a tighter question than the specification proposed, but answers it with rigour that a broader scope would have precluded.

5.2 Design and Implementation Rationale

Plain-backbone detection architecture. The projection pyramid follows the approach of Li et al. [24], who established that a non-hierarchical vision transformer works as a detection backbone with only a simple feature pyramid built from a single-scale feature map. This was chosen over remaking a hierarchical backbone because it isolates the variable being inspected.

Reuse of the Ultralytics neck and head. Sharing the C3k2, Conv and Detect modules between families ensures that the only differences between them lie in the backbone and the adaptation strategy. Developing a tailored detection head would have introduced a second uncontrolled variable.

COCO-pretrained baselines. Both families start from large-scale pretraining. Training the baselines from random initialisation would have produced a larger apparent margin and an invalid comparison, as the study concerns the pretraining corpus and the adaptation strategy rather than the presence or absence of pretraining.

Per-architecture baseline tuning. RT-DETR required different hyperparameters from the YOLO family because of its different loss formulation. Applying YOLO defaults to it would have created a false impression of weakness in the strongest transformer-based baseline.

GroupNorm in place of BatchNorm. Adopted to address instability measured during experimentation rather than chosen in advance, as described in Section 3.5.

Hyperparameter search design. Two parameters were varied: unfrozen block depth and the backbone-to-head learning rate ratio. Input resolution, augmentation policy, schedule length and clipping threshold were held constant across the grid so that differences could be attributed cleanly to the two parameters under investigation. A broader search space could not be explored given the per-epoch cost documented in Section 4.1.

5.3 Lessons Learned

Measurement faults resemble model faults. Each of the three issues encountered, EMA evaluation lag, BatchNorm instability, and the serialisation fault, initially presented as poor model performance. Time was spent investigating model architecture and hyperparameters before the measurement path was suspected. The core takeaway is to verify and track the metric before attempting to optimise it.

Published data is not verified data. WTBs2025 is published in a peer-reviewed data journal under a title describing it as a high-quality dataset, yet two of its categories contain substantial labelling error. Had this not been checked, results would have been reported on partly meaningless labels with no indication that anything was wrong.

Notebook state is a hazard. The implicit retention of state across cells contributed to the difficulty of diagnosing the checkpoint fault, where an evaluation cell referenced a model class defined in a cell that had since been deleted. In the future, moving stable

code into version-controlled modules imported by the notebooks would eliminate this kind of problem.

5.4 Evaluation of the Outcome

What was achieved. A methodologically matched comparison across five datasets, eighteen conventional architectures and forty-five foundation-model configurations, in both per-dataset and merged cross-dataset settings. A data-volume threshold identified and supported by two independent lines of evidence. A cross-dataset result in which the foundation model outperforms the conventional baseline on every individual test set. A verification methodology that identified substantial annotation error in a published dataset. And three measurement faults found and corrected rather than reported through.

What the results support. The threshold finding is the most transferable outcome: foundation-model adaptation in this domain requires a minimum annotation volume, below which a smaller architecture fine-tuned from general pretraining is the better choice. The cross-dataset result supports a secondary claim that broad pretraining allows tolerance of source heterogeneity.

What the results do not support. Single-seed reporting means the margin on the smallest dataset sits within observed checkpoint variance and should be read as indicative rather than established. The merged baseline was a single architecture rather than the full sweep, so the merged margin is not measured against an equivalently tuned comparison. And with only one backbone evaluated, it is not established whether the threshold is a property of AM-RADIO specifically or of foundation-model fine-tuning for detection more generally.

What would be done differently. Per-epoch cost would be measured before scope was committed to. Checkpoint serialisation would be verified from the first run rather than the last. Annotation verification would be budgeted for explicitly rather than discovered as a necessity. And compute would be reserved for multi-seed replication of the headline result, which would have converted the strongest finding from indicative to established.

References

- [1] A. Shihavuddin et al., "Wind turbine surface damage detection by deep learning aided drone inspection analysis," *Energies*, vol. 12, no. 4, p. 676, 2019.
- [2] A. Foster, O. Best, M. Gianni, A. Khan, K. Collins, and S. Sharma, "Drone footage wind turbine surface damage detection," in *Proc. IEEE 14th IVMSP Workshop*, Nafplio, Greece, 2022, pp. 1–5.
- [3] I. Gohar, J. See, A. Halimi, and W. K. Yew, "Automatic defect detection in wind turbine blade images: Model benchmarks and re-annotations," in *Proc. IEEE ICMEW*, 2023, pp. 290–295.
- [4] I. Gohar, A. Halimi, J. See, W. K. Yew, and C. Yang, "Slice-aided defect detection in ultra high-resolution wind turbine blade images," *Machines*, vol. 11, no. 10, p. 953, 2023.
- [5] C. Yang, X. Liu, H. Zhou, Y. Ke, and J. See, "Towards accurate image stitching for drone-based wind turbine blade inspection," *Renew. Energy*, vol. 203, pp. 267–279, 2023.
- [6] L. Ji, J. Cheng, and S. Wu, "Multiclass dataset for intelligent detection of wind turbine blade defects using drone imagery," *Sci. Data*, vol. 13, no. 1, p. 396, 2026.
- [7] R. Zhang, S. Wang, S. Liu, Y. Liang, T. Wang, C. Liu, and S. Chen, "Wind turbine blades surface defect high-quality image dataset construction and performance validation," *Sci. Data*, 2026, doi: 10.1038/s41597-026-07455-1.
- [8] H. Yu, J. G. Wang, Y. X. Han, B. Fan, and C. Zhang, "Research on an intelligent identification method for wind turbine blade damage based on CBAM-BiFPN-YOLOv8," *Processes*, vol. 12, no. 1, p. 205, 2024.
- [9] L. Zou, A. Chen, C. Li, X. Yang, and Y. Sun, "DCW-YOLO: An improved method for surface damage detection of wind turbine blades," *Appl. Sci.*, vol. 14, no. 19, p. 8763, 2024.
- [10] Y. He, X. B. Niu, C. P. Hao, Y. F. Li, L. Kang, and Y. Wang, "An adaptive detection approach for multi-scale defects on wind turbine blade surface," *Mech. Syst. Signal Process.*, vol. 219, p. 111592, 2024.
- [11] W. Li, W. Zhao, and Y. Du, "Large-scale wind turbine blade operational condition monitoring based on UAV and improved YOLOv5 deep learning model," *Mech. Syst. Signal Process.*, vol. 226, p. 112386, 2025.
- [12] J. Guo, C. Liu, J. Cao, and D. Jiang, "Damage identification of wind turbine blades with deep convolutional neural networks," *Renew. Energy*, vol. 174, pp. 122–133, 2021.
- [13] W. Li, Z. Pan, Q. Zhu, and Y. Du, "Wind turbine blade defect detection and measurement technology based on improved SegFormer and pixel matching," *Opt. Laser Technol.*, vol. 179, p. 111381, 2024.
- [14] D. Dwivedi, K. V. S. M. Babu, P. K. Yemula, P. Chakraborty, and M. Pal, "Identification of surface defects on solar PV panels and wind turbine blades using attention based deep learning model," *Eng. Appl. Artif. Intell.*, vol. 131, p. 107836, 2024.
- [15] X. Jia and X. Chen, "AI-based optical-thermal video data fusion for near real-time blade segmentation in normal wind turbine operation," *Eng. Appl. Artif. Intell.*, vol. 127, p. 107325, 2024.
- [16] M. Memari, M. Shekaramiz, M. A. S. Masoum, and A. C. Seibi, "Data fusion and ensemble learning for advanced anomaly detection using multi-spectral RGB and thermal imaging of small wind turbine blades," *Energies*, vol. 17, no. 3, 2024.
- [17] P. Rizk, F. Rizk, S. S. Karganroudi, A. Ilinca, R. Younes, and J. Khoder, "Advanced wind turbine blade inspection with hyperspectral imaging and 3D convolutional neural networks for damage detection," *Energy AI*, vol. 16, 2024.
- [18] L. Deng, Y. Guo, and B. Chai, "Defect detection on a wind turbine blade based on digital image processing," *Processes*, vol. 9, no. 8, 2021.

- [19] I. Gohar, W. K. Yew, A. Halimi, and J. See, "Review of state-of-the-art surface defect detection on wind turbine blades through aerial imagery: Challenges and recommendations," *Eng. Appl. Artif. Intell.*, vol. 144, p. 109970, 2025.
- [20] G. Jocher, J. Qiu, and A. Chaurasia, "Ultralytics YOLO," Zenodo, 2025. doi: 10.5281/zenodo.14820090. Accessed: Sep. 10, 2026.
- [21] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," in *Proc. ECCV*, 2024, pp. 1–21.
- [22] A. Wang, H. Chen, L. Liu, *et al.*, "YOLOv10: Real-time end-to-end object detection," in *Proc. NeurIPS*, 2024.
- [23] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs beat YOLOs on real-time object detection," in *Proc. IEEE/CVF CVPR*, Seattle, WA, USA, 2024.
- [24] Y. Li, H. Mao, R. Girshick, and K. He, "Exploring plain vision transformer backbones for object detection," in *Proc. ECCV*, 2022, pp. 280–296.
- [25] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. ICML*, 2021, pp. 8748–8763.
- [26] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging properties in self-supervised vision transformers," in *Proc. IEEE/CVF ICCV*, 2021, pp. 9650–9660.
- [27] M. Oquab *et al.*, "DINOv2: Learning robust visual features without supervision," *Trans. Mach. Learn. Res.*, 2024.
- [28] A. Kirillov *et al.*, "Segment Anything," in *Proc. IEEE/CVF ICCV*, 2023, pp. 4015–4026.
- [29] M. Ranzinger, G. Heinrich, J. Kautz, and P. Molchanov, "AM-RADIO: Agglomerative vision foundation model reduce all domains into one," in *Proc. IEEE/CVF CVPR*, 2024, pp. 12490–12500.
- [30] G. Heinrich, M. Ranzinger, H. Yin, Y. Lu, A. Tao, B. Catanzaro, P. Molchanov, and J. Kautz, "RADIOv2.5: Improved baselines for agglomerative vision foundation models," in *Proc. IEEE/CVF CVPR*, 2025.
- [31] N. Cohen and Y. Hoshen, "Sub-image anomaly detection with deep pyramid correspondences," *arXiv:2005.02357*, 2020.
- [32] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "PaDiM: A patch distribution modeling framework for anomaly detection and localization," in *Proc. ICPR Workshops*, 2020, pp. 475–489.
- [33] K. Roth *et al.*, "Towards total recall in industrial anomaly detection," in *Proc. ECCV*, 2022.
- [34] G. Xie, J. Wang, J. Liu, F. Zheng, and Y. Jin, "Pushing the limits of few-shot anomaly detection in industry vision: GraphCore," *arXiv:2301.12082*, 2023.
- [35] T. Reiss, N. Cohen, E. Horwitz, R. Abutbul, and Y. Hoshen, "Anomaly detection requires better representations," in *Proc. ECCV Workshops*, 2022, pp. 56–68.
- [36] S. Damm, M. Laszkiewicz, J. Lederer, and A. Fischer, "AnomalyDINO: Boosting patch-based few-shot anomaly detection with DINOv2," in *Proc. IEEE/CVF WACV*, 2025.
- [37] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, and C. Steger, "The MVTec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection," *Int. J. Comput. Vis.*, vol. 129, no. 4, pp. 1038–1059, 2021.
- [38] Y. Zou, J. Jeong, L. Pemula, D. Zhang, and O. Dabeer, "Spot-the-difference self-supervised pre-training for anomaly detection and segmentation," in *Proc. ECCV*, 2022, pp. 392–408.

- [39] M. A. M. Eltoum, E. Iqbal, S. U. Khan, and Y. Abdulrahman, "RadioCore: Few-shot industrial anomaly segmentation with multi-scale Radio ViT features," in *Proc. IEEE/CVF CVPRW, VISION Workshop*, 2026, pp. 7288–7297.
- [40] E. Iqbal, S. U. Khan, S. Javed, B. Moyo, Y. Zweiri, and Y. Abdulrahman, "Multi-scale feature reconstruction network for industrial anomaly detection," *Knowl.-Based Syst.*, vol. 305, p. 112650, 2024.
- [41] J. Jeong, Y. Zou, T. Kim, D. Zhang, A. Ravichandran, and O. Dabeer, "WinCLIP: Zero-/few-shot anomaly classification and segmentation," in *Proc. IEEE/CVF CVPR*, 2023, pp. 19606–19616.
- [42] X. Chen, Y. Han, and J. Zhang, "A zero-/few-shot anomaly classification and segmentation method for CVPR 2023 VAND workshop challenge tracks 1 & 2," *arXiv:2305.17382*, 2023.
- [43] X. Li, Z. Huang, X. Feng, and Y. Zhou, "MuSc: Zero-shot industrial anomaly classification and segmentation with mutual scoring of the unlabeled images," in *Proc. ICLR*, 2024.
- [44] Q. Zhou, G. Pang, Y. Tian, S. He, and J. Chen, "AnomalyCLIP: Object-agnostic prompt learning for zero-shot anomaly detection," in *Proc. ICLR*, 2024.
- [45] P. Zajec, J. M. Rožanec, S. Theodoropoulos, M. Fontul, E. Koehorst, B. Fortuna, and D. Mladeníć, "Few-shot learning for defect detection in manufacturing," *Int. J. Prod. Res.*, vol. 62, no. 19, pp. 6979–6998, 2024.